

Derick Beckwith  
COINS 496 Senior Portfolio  
April 11, 2008

Program 1: Illustrate array processing algorithms

Class: COINS 315 – Data Structures

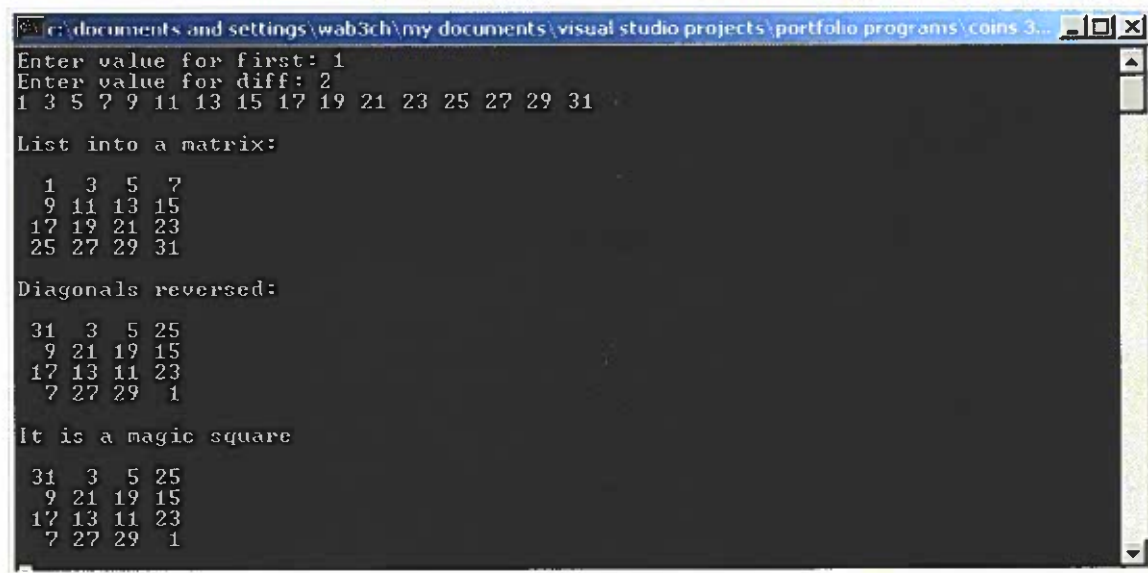
Grade Received: A

Programming Language: C++

Operating Systems: Windows

Compiler: Visual C++ 7.1

## Program 1 screenshot:



```
c:\documents and settings\wab3ch\my documents\visual studio projects\portfolio programs\coins 3...
Enter value for first: 1
Enter value for diff: 2
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31

List into a matrix:

  1  3  5  7
  9 11 13 15
 17 19 21 23
 25 27 29 31

Diagonals reversed:

31  3  5 25
 9 21 19 15
17 13 11 23
 7 27 29  1

It is a magic square

31  3  5 25
 9 21 19 15
17 13 11 23
 7 27 29  1
```

```
// -----  
//  
// Student Name:      Derick Beckwith  
// Program Name:      Program1  
// Creation Date:      31 August 2003  
// Last Modified Date: April 11, 2008  
// COINS Course:      COINS 315 C++ Data Structures  
// Grade Received:     A  
// Purpose:            Illustrate array processing algorithms.  
// Filename:           "program1all.cpp"  
// -----  
//  
#include <iostream>  
#include <iomanip>      // for setw() in printMatrix()  
using namespace std;  
  
// global named constants  
// note: change value of ROWS and COLS here to change size of arrays used in program  
const int ROWS = 4;  
const int COLS = 4;  
const int ARRAY_SIZE = ROWS * COLS;  
  
// type definitions using global named constants  
typedef int matrixType[ROWS][COLS];  
typedef int listmatrixType[ARRAY_SIZE];  
  
// <----- begin funtion prototypes ----->  
  
// fills an array with a sequence of integers  
void createArithmeticSeq(listmatrixType seq, const int arraysize);  
  
// places contents of a one-dimensional array into a two-dimensional matrix  
void matricize(matrixType mymatrix, listmatrixType mylist, const int rows, const int  
    cols);  
  
// reverse both diagonals of a matrix  
void reverseDiagonals(matrixType mymatrix, const int rows);  
  
// output to the screen in as square a form as possible  
void printMatrix(matrixType matrix, int rows, int cols, int fieldwidth);  
  
// Magic Square functions:  
  
// determines if mymatrix is a magic square  
void magicCheck(matrixType mymatrix, listmatrixType mylist, const int listcount,  
    const int rows, const int cols);  
  
// checks one row or one column at a time  
bool comparerow(matrixType mymatrix, const int row, const int cols, const int magicnum);  
bool comparecol(matrixType mymatrix, const int rows, const int col, const int magicnum);  
  
// uses above functions to check all the rows or all the columns  
bool comparerows(matrixType mymatrix, const int rows, const int cols, const int magicnum)  
    ;  
bool comparecols(matrixType mymatrix, const int rows, const int cols, const int magicnum)  
    ;  
  
// check both diagonals  
bool comparediags(matrixType mymatrix, const int rows, const int cols, const int  
    magicnum);  
  
// <----- end function prototypes ----->
```

```
int main()
{
    listmatrixType mylist;
    matrixType mymatrix;

    // a.) create the arithmetic sequence
    createArithmeticSeq(mylist, ARRAY_SIZE);

    // b.) put contents of array into a matrix
    matricize(mymatrix, mylist, ROWS, COLS);
    cout << endl << "List into a matrix: " << endl;
    printMatrix(mymatrix, ROWS, COLS, 3);

    // c.) reverse the diagonals
    reverseDiagonals(mymatrix, ROWS);
    cout << endl << "Diagonals reversed: " << endl;
    printMatrix(mymatrix, ROWS, COLS, 3);

    // d.) check for a magic square
    magicCheck(mymatrix, mylist, ARRAY_SIZE, ROWS, COLS);

    // e.) print the matrix in a square form
    printMatrix(mymatrix, ROWS, COLS, 3);

    int x;
    cin >> x;

    return 0;
}

void createArithmeticSeq(listmatrixType seq, const int arraysize)
{
    int x, first, diff;          // loop indexing

    // get values for first and diff from user
    cout << "Enter value for first: ";
    cin >> first;
    cout << "Enter value for diff: ";
    cin >> diff;

    // assert value of first element
    seq[0] = first;
    cout << seq[0] << " ";

    // fill rest of array with number sequence
    for(x = 1; x < arraysize; x++)
    {
        seq[x] = first + (x * diff);
        cout << seq[x] << " ";
    }

    cout << endl;
}

void matricize(matrixType mymatrix, listmatrixType mylist, const int rows, const int cols)
{
    int row, col, x = 0;

    for(row = 0; row < rows; row++)
    {
        for(col = 0; col < cols; col++)
        {
            mymatrix[row][col] = mylist[x];
            x++;
        }
    }
}
```

```
    }  
}  
  
void reverseDiagonals(matrixType mymatrix, const int rows)  
{  
    int row, temp;  
  
    // reverse main diagonal  
    for(row = 0; row < rows / 2; row++)  
    {  
        temp = mymatrix[row][row];  
        mymatrix[row][row] = mymatrix[rows-1-row][rows-1-row];  
        mymatrix[rows-1-row][rows-1-row] = temp;  
    }  
  
    // reverse the opposite diagonal  
    for(row = 0; row < rows / 2; row++)  
    {  
        temp = mymatrix[row][rows-1-row];  
        mymatrix[row][rows-1-row] = mymatrix[rows-1-row][row];  
        mymatrix[rows-1-row][row] = temp;  
    }  
}  
  
void printMatrix(matrixType matrix, int rows, int cols, int fieldwidth)  
{  
    int row, col, fieldsize = 0;  
  
    cout << endl;  
  
    // output elements in matrix in a square form  
    for(row = 0; row < rows; row++)  
    {  
        for(col = 0; col < cols; col++)  
        {  
            cout << setw(fieldwidth) << matrix[row][col];  
        }  
        cout << endl;  
    }  
}  
  
void magicCheck(matrixType mymatrix, listmatrixType mylist, const int listcount,  
                const int rows, const int cols)  
{  
    int x, magicnum, sum = 0;  
  
    // determine magic number from one-dimensional array mylist  
    for(x = 0; x < listcount; x++)  
    {  
        sum += mylist[x];  
    }  
  
    magicnum = sum / rows;  
  
    // test square matrix against magic number  
    if (true == comparediags(mymatrix, rows, cols, magicnum) &&  
        true == comparerows(mymatrix, rows, cols, magicnum) &&  
        true == comparecols(mymatrix, rows, cols, magicnum))  
    {  
        cout << endl << "It is a magic square" << endl;  
    }  
    else  
    {  
        cout << endl << "It is not a magic square" << endl;  
    }  
}
```

```
bool comparerows(matrixType mymatrix, const int rows, const int cols, const int magicnum)
{
    int row;
    for(row = 0; row < rows; row++)
    {
        // short-circuit at first row that fails test
        if (false == (comparerow(mymatrix, row, cols, magicnum)))
        {
            return false;
        }
    }

    return true;
}

bool comparerow(matrixType mymatrix, const int row, const int cols, const int magicnum)
{
    int col, sum = 0;

    for(col = 0; col < cols; col++)
    {
        sum += mymatrix[row][col];
    }

    if(sum != magicnum)
    {
        return false;
    }

    return true;
}

bool comparecols(matrixType mymatrix, const int rows, const int cols, const int magicnum)
{
    int col;
    for(col = 0; col < cols; col++)
    {
        // short-circuit at first column that fails test
        if (false == (comparecol(mymatrix, rows, col, magicnum)))
        {
            return false;
        }
    }

    return true;
}

bool comparecol(matrixType mymatrix, const int rows, const int col, const int magicnum)
{
    int row, sum = 0;

    for(row = 0; row < rows; row++)
    {
        sum += mymatrix[row][col];
    }

    if(sum != magicnum)
    {
        return false;
    }

    return true;
}
```

```
bool comparediags(matrixType mymatrix, const int rows, const int cols, const int
magicnum)
{
    int row, sum = 0;

    // sum main diagonal
    for(row = 0; row < rows; row++)
    {
        sum += mymatrix[row][row];
    }

    // compare sum of main diagonal to magic number
    if(sum != magicnum)
    {
        return false;
    }

    sum = 0;

    // sum opposite diagonal
    for(row = 0; row < rows; row++)
    {
        sum += mymatrix[row][(rows - 1) - row];
    }

    // compare sum of opposite diagonal to magic number
    if(sum != magicnum)
    {
        return false;
    }

    return true;
}
```



Program 2: Screen Saver

Class: COINS 325 – Java Programming I

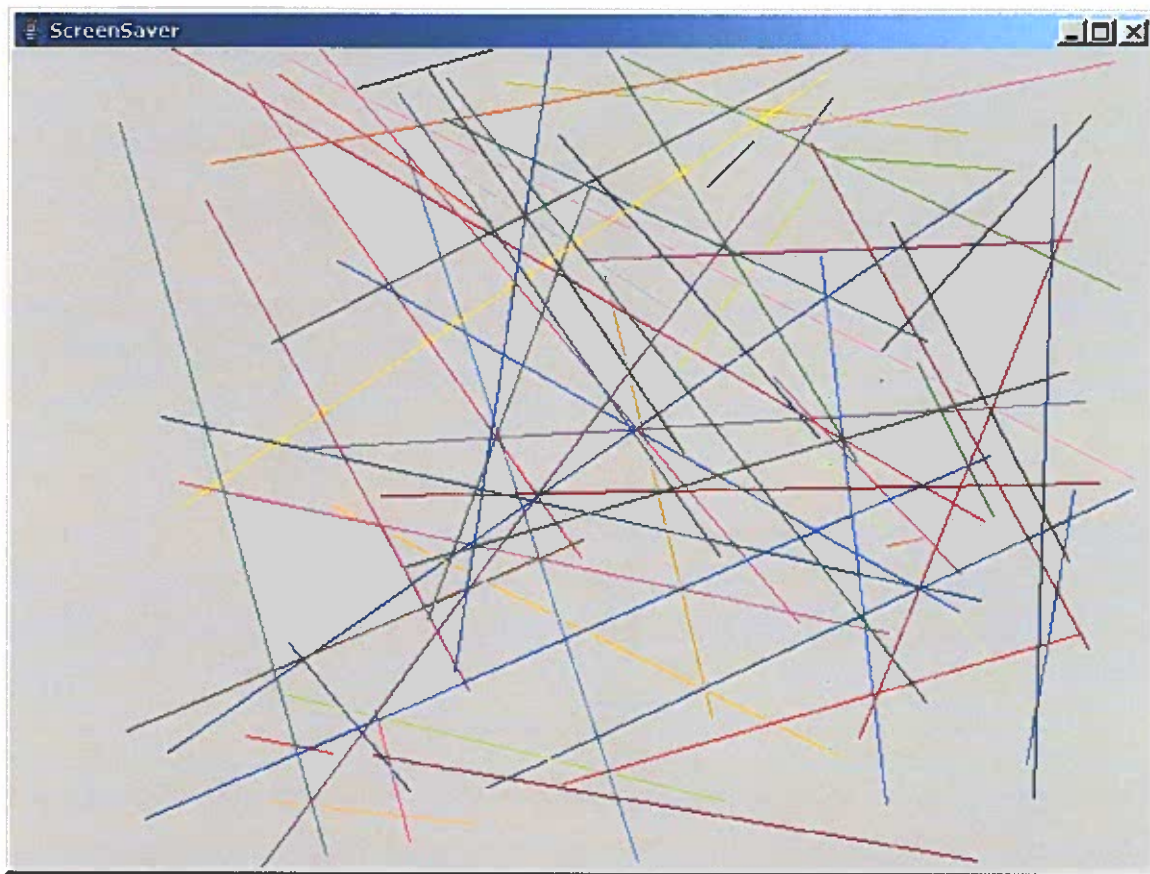
Grade Received: A

Programming Language: Java

Operating Systems: Windows running JVM v1.3.1\_09

Compiler: Java 2 SDK, SE v1.4.2\_13

Program 2 screenshot:





```
        ( float ) Math.random() ) );

    // draw a line with specified random color
    // between the two random endpoints
    g.drawLine( x, y, xl, yl );

    // build in a delay to allow drawing of lines to be seen
    for ( int j = 0; j < pause; j++ )
    {
        ;
    }

    // clear the lines to start drawing again
    repaint();
}

} // end class ScreenSaver
```

### Program 3: Peer-to-Peer Application

Class: COINS 332 – Applied Networking

Grade Received: A

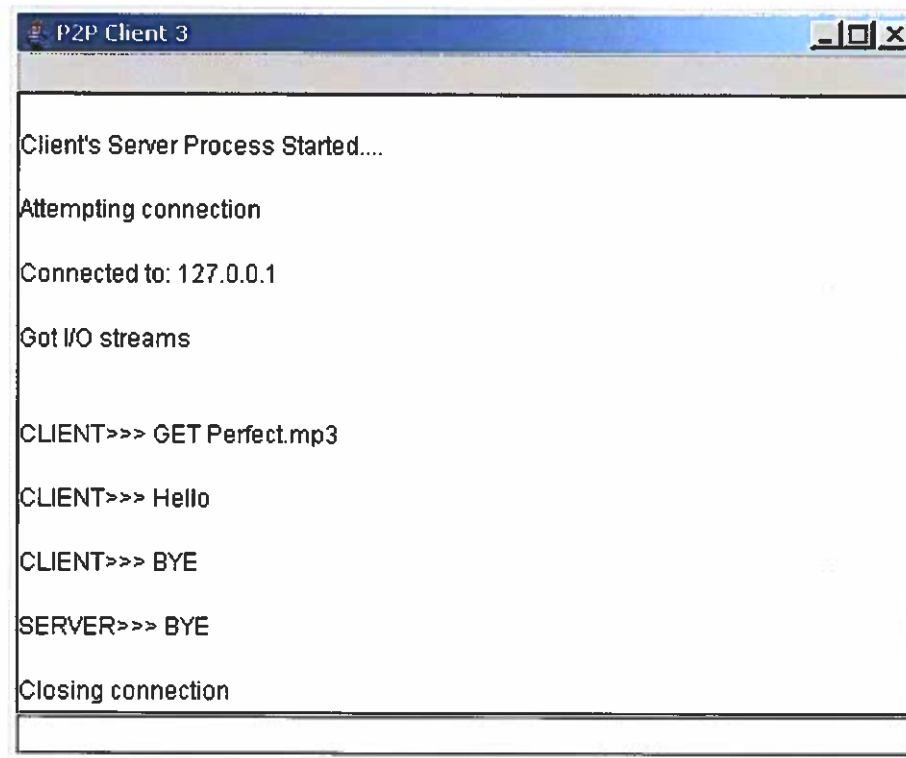
Programming Language: Java

Operating Systems: Windows running JVM v1.3.1\_09

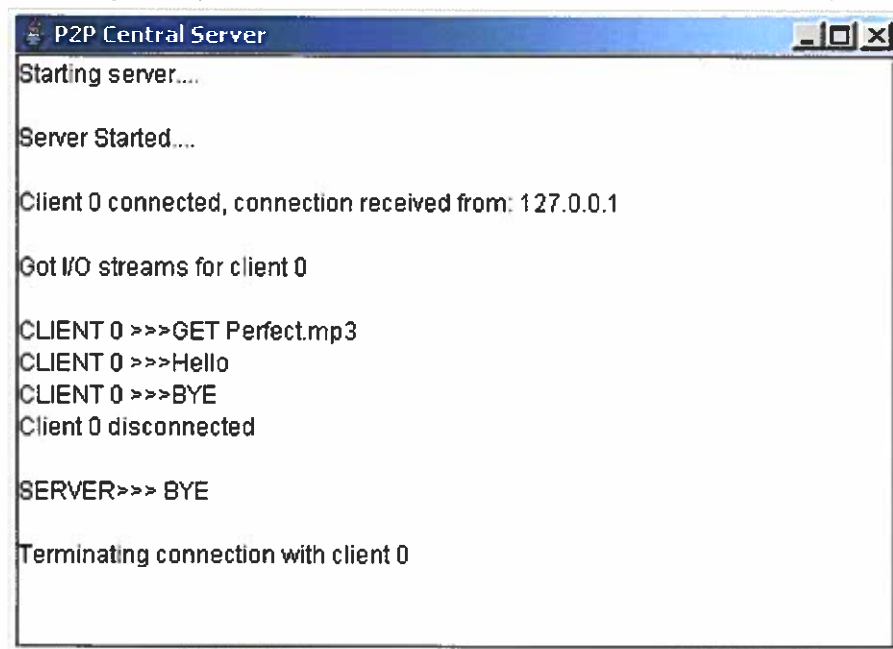
Compiler: Java 2 SDK, SE v1.4.2\_13

## Program 3 screenshots:

### P2P Client:



### P2P Server:



```
// -----  
//  
// Student Name:      Derick Beckwith  
// Program Name:      Peer-to-Peer Client  
// Creation Date:      March 1, 2005  
// Last Modified Date: April 19, 2005  
// COINS Course:      COINS 332 Applied Networking  
// Grade Received:     A  
// Purpose:            P2P client that sends and receives  
//                     information sent from a central P2P Server.  
// Filename:           Client.java  
// -----  
  
import java.io.*;  
import java.net.*;  
import java.util.*;  
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;  
import javax.swing.event.*;  
  
public class Client extends JFrame  
{  
    private JTextField enterField, downloadField;  
    private JTextArea displayArea;  
  
    // for central server i/o  
    private ObjectOutputStream output;  
    private ObjectInputStream input;  
  
    // for parsing  
    private StringTokenizer tokenizedMessage;  
    private String firstToken;  
    private String chatServer;  
    private Socket client;      // for connection to central server  
    private Server server;  
  
    // initialize Client and set up GUI  
    public Client( String host )  
    {  
        super( "P2P Client 3" );  
  
        // set P2P Central Server to which this client connects  
        chatServer = host;  
        // create a new internal Server class to handle sending files to requesting clients  
        server = new Server(this);  
  
        Container container = getContentPane();  
  
        // create enterField and register listener  
        enterField = new JTextField();  
        enterField.setEditable( false );  
        enterField.addActionListener(  
            new ActionListener()  
            {  
                // send message to central server  
                public void actionPerformed((ActionEvent event) )  
                {  
                    sendData( event.getActionCommand() );  
                    enterField.setText( "" );  
                }  
            }  
        );  
    }  
};
```

```
// create download field and register listener
downloadField = new JTextField();
downloadField.addActionListener(
    new ActionListener()
    {
        public void actionPerformed( ActionEvent event )
        {
            download( event.getActionCommand() );
        }
    }
);

// attach GUI components to the JFrame
container.add( enterField, BorderLayout.NORTH );
container.add( downloadField, BorderLayout.SOUTH );

// create displayArea
displayArea = new JTextArea();
container.add( new JScrollPane( displayArea ), BorderLayout.CENTER );

setSize( 600, 350 );
setVisible( true );

} // end Client constructor

// connect to server and process messages from server
private void runClient()
{
    // connect to server, get streams, process connection
    try
    {
        server.start();
        connectToServer();
        getStreams();
        processConnection();
    }
    // server closed connection
    catch ( EOFException eofException )
    {
        System.err.println( "Client terminated connection" );
    }
    // process problems communicating with server
    catch ( IOException ioException )
    {
        ioException.printStackTrace();
    }
    // Close connection
    finally
    {
        closeConnection();
    }
}

// end method runClient

private void download( String requestLine )
{
    Downloader d = new Downloader( this, requestLine );
    d.start();
}

// connect to server
private void connectToServer() throws IOException
{
    displayMessage( "Attempting connection\n" );

    // create Socket to make connection to server
```



```
    client = new Socket( InetAddress.getByName( chatServer ), 5001 );

    // display connection information
    displayMessage( "Connected to: " + client.getInetAddress().getHostName() );
}

// get streams to send and receive data
private void getStreams() throws IOException
{
    // set up output stream for objects
    output = new ObjectOutputStream( client.getOutputStream() );
    output.flush(); // flush output buffer to send header information

    // set up input stream for objects
    input = new ObjectInputStream( client.getInputStream() );

    displayMessage( "\nGot I/O streams\n" );
}

// process connection with server
private void processConnection() throws IOException
{
    // enable enterField so client user can send messages
    setTextFieldEditable( true );

    // process messages sent from server
    do
    {
        // read message and display it
        try
        {
            String message = "Connection OK";
            message = ( String ) input.readObject();
            displayMessage( "\nSERVER>>> " + message );
            tokenizedMessage = new StringTokenizer( message );
            firstToken = tokenizedMessage.nextToken();
        }
        // catch problems reading from client
        catch ( ClassNotFoundException classNotFoundException )
        {
            displayMessage( "\nUnknown object type received" );
        }

        } while ( !firstToken.equals("BYE") );
    } // end method processConnection

// close streams and socket
private void closeConnection()
{
    displayMessage( "\nClosing connection" );
    setTextFieldEditable( false ); // disable enterField

    try
    {
        output.close();
        input.close();
        client.close();
    }
    catch( IOException ioException )
    {
        ioException.printStackTrace();
    }
}
```

```
// send message to server
private void sendData( String message )
{
    // send object to server
    try
    {
        output.writeObject( message );
        output.flush();
        displayMessage( "\nCLIENT>>> " + message );
    }
    // process problems sending object
    catch ( IOException ioException )
    {
        displayArea.append( "\nError writing object" );
    }
}

// utility method called from other threads to manipulate
// displayArea in the event-dispatch thread
public void displayMessage( final String messageToDisplay )
{
    // display message from GUI thread of execution
    SwingUtilities.invokeLater(
        new Runnable() { // inner class to ensure GUI updates properly

            public void run() // updates displayArea
            {
                displayArea.append( "\n" + messageToDisplay );
                displayArea.setCaretPosition(
                    displayArea.getText().length() );
            }

        } // end inner class
    ); // end call to SwingUtilities.invokeLater
}

// utility method called from other threads to manipulate
// enterField in the event-dispatch thread
private void setTextFieldEditable( final boolean editable )
{
    // display message from GUI thread of execution
    SwingUtilities.invokeLater(
        new Runnable() { // inner class to ensure GUI updates properly

            public void run() // sets enterField's editability
            {
                enterField.setEditable( editable );
            }

        } // end inner class
    ); // end call to SwingUtilities.invokeLater
}

public static void main( String args[] )
{
    Client application;

    if ( args.length == 0 )
    {
        application = new Client( "127.0.0.1" );
    }
    else
    {
        application = new Client( args[ 0 ] );
    }
}
```

```

    }

    application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
    application.runClient();
}

} // end class Client

// downloader is an inner class responsible for retrieving a file from another peer
class Downloader extends Thread
{
    private Socket peer;           // for connection to another peer
    private ObjectOutputStream outToPeer;
    private ObjectInputStream inFromPeer;

    // for saving downloaded files to disk
    private FileOutputStream outToFile;
    Client control;

    private StringTokenizer tokenizedMessage;
    private StringTokenizer header;
    private String fileRequest;
    private String response;
    private String path;
    private String method;
    private String peerIP;
    private String port;
    private String fileName;

    public Downloader( Client c, String requestLine )
    {
        control = c;
        fileRequest = requestLine;

        tokenizedMessage = new StringTokenizer( requestLine );
        method = tokenizedMessage.nextToken();
        fileName = tokenizedMessage.nextToken();
        peerIP = tokenizedMessage.nextToken();
        port = tokenizedMessage.nextToken();

        displayMessage( "Trying to Connect to " + peerIP + " on port " + port );

        try
        {
            connectToPeer();
            getStreams();
        }
        catch( IOException ioException )
        {
            displayMessage( "Could not connect to peer");
            ioException.printStackTrace();
            System.exit( 1 );
        }
    }

    private void connectToPeer() throws IOException
    {
        // connect to other peer's server process
        displayMessage( "Attempting connection to peer " + peerIP + ". . .\n" );
        peer = new Socket( InetAddress.getLocalHost(), Integer.parseInt(port) );
    }

    private void getStreams() throws IOException
    {
        outToPeer = new ObjectOutputStream( peer.getOutputStream() );
        outToPeer.flush();
    }
}

```

```
        inFromPeer = new ObjectInputStream( peer.getInputStream() );
    }

    private void displayMessage( final String messageToDisplay )
    {
        control.displayMessage( messageToDisplay );
    }

    // close streams and socket
    private void closeConnection()
    {
        displayMessage( "\nTerminating connection with client peer\n" );

        try
        {
            outToPeer.close();
            inFromPeer.close();
            peer.close();
        }
        catch( IOException ioException )
        {
            ioException.printStackTrace();
        }
    }

    // send message to client
    private void sendData( String message )
    {
        // send object to client
        try
        {
            outToPeer.writeObject( message );
            outToPeer.flush();
            displayMessage( "\nPEER>>> " + message );
        }
        // process problems sending object
        catch ( IOException ioException )
        {
            displayMessage( "\nError writing object" );
        }
    }

    public void run()
    {
        displayMessage("Connected to peer " + peerIP );
        displayMessage("sending data to peer" );
        sendData( fileRequest );
        //closeConnection();

        displayMessage("Reading sponse from peer " + peerIP );
        try
        {
            response = ( String ) inFromPeer.readObject();
        }
        catch ( IOException ioException )
        {
            displayMessage("Error reading peer's response!");
        }
        catch ( ClassNotFoundException classNotFoundException )
        {
            displayMessage( "\nUnknown object type received" );
        }

        header = new StringTokenizer( response );
        // should be "Content-Length:"
        String contentLength = header.nextToken();
    }
}
```

```
// get number of bytes to create byte array of right length
int numOfBytes = Integer.parseInt( header.nextToken() );

displayMessage("Creating array of " + numOfBytes + " bytes to hold file " +
fileName );
byte[] fileInBytes = new byte[ numOfBytes ];

try
{
    inFromPeer.read( fileInBytes );
}
catch ( IOException ioException )
{
    displayMessage("Error reading file from socket!");
}

path = "C:\\downloaded files\\Perfect.mp3";
displayMessage("Creating a new FileOutputStream class and writing to file: "
+ path );

try
{
    outToFile = new FileOutputStream( path );
    outToFile.write( fileInBytes );
}
catch ( IOException ioException )
{
    displayMessage("Error writing the file: " + path + " to disk!");
}
finally
{
    displayMessage("Completed downloading the file: " + path);
}

try
{
    outToFile.close();
}
catch ( IOException ioException )
{
    displayMessage("Error closing the file: " + path);
}

} // end method run

} // end class Downloader

class Server extends Thread
{
    private ServerSocket server;
    private User users[];
    private int users_count;
    Client control;

    public Server( Client c )
    {
        control = c;
        users_count = 0;
        users = new User[ 10 ];
        try
        {
            server = new ServerSocket( 6999, 10 );
        }
        catch( IOException ioException )
        {
        }
    }
}
```

```

        ioException.printStackTrace();
        System.exit( 1 );
    }
}

public void run()
{
    control.displayMessage("Client's Server Process Started...\n");
    while ( users_count < 10)
    {
        try
        {
            users[ users_count ] = new User( server.accept(), users_count );
            users[ users_count ].start();
        }
        catch( IOException ioException )
        {
            ioException.printStackTrace();
            System.exit( 1 );
        }
        users_count++;
    }
}

// User is an inner class that is spawned as a new thread
// for each connecting client to the Server class
private class User extends Thread
{
    private Socket connection;
    private ObjectInputStream input;
    private ObjectOutputStream output;
    private int userNumber;
    private String usersIP;
    private StringTokenizer tokenizedMessage;
    private String firstToken;
    private String getFileName;

    public User( Socket socket, int number )
    {
        userNumber = number;
        connection = socket;
        try
        {
            getStreams();
        }
        catch( IOException ioException )
        {
            ioException.printStackTrace();
            System.exit( 1 );
        }
    } // end User

    // get streams to send and receive data
    private void getStreams() throws IOException
    {
        usersIP = connection.getInetAddress().toString(); // retrieve IP address of
client
        usersIP = usersIP.substring( 1, usersIP.length() ); // trim off the "/" slash

        displayMessage( "Client peer " + userNumber + " connected, connection
received from: " + usersIP );
        // set up output stream for objects
        output = new ObjectOutputStream( connection.getOutputStream() );
        output.flush(); // flush output buffer to send header information
        // set up input stream for objects
        input = new ObjectInputStream( connection.getInputStream() );
    }
}

```



```

        if ( file.exists() && file.isFile() )
        {
            int numofBytes = ( int ) file.length();
            FileInputStream inFile = new FileInputStream( getFileName );
            byte[] fileInBytes = new byte[ numofBytes ];

            try
            {
                inFile.read( fileInBytes );
            }
            catch ( IOException ioException )
            {
                message = "Error reading requested file: " + getFileName;
                sendData( message );
                displayMessage( message );
            }

            // send header and content data to requesting peer
            sendData( "Content-Length: " + numofBytes + "\r\n" );

            try
            {
                output.write( fileInBytes, 0, numofBytes );
                output.flush();
            }
            catch ( IOException ioException )
            {
                message = "Error sending requested file: " + getFileName;
                sendData( message );
                displayMessage( message );
            }
        }
        else // file does not exist on the local system
        {
            message = "Error! Requested File: " + getFileName + " Not
Found!";
            sendData( message );
            displayMessage( message );
        }
    }
    else
    {
        message = "Error: Invalid Client Request!";
        sendData( message );
    }
} // end if/else
} // end try
// catch problems reading from client
catch ( ClassNotFoundException classNotFoundException )
{
    displayMessage( "\nUnknown object type received" );
}

catch( IOException ioException )
{
    ioException.printStackTrace();
    System.exit( 1 );
}

//} while( !firstToken.equals("BYE") );

displayMessage("Client peer " + userNumber + " disconnected");
sendData( "BYE" );
closeConnection();
users_count--; // after a user quits its connection, decrease the
number of users

```



```
        } // end run method
    } // end class User
} // end class Server
```

```
// -----  
//  
// Student Name:      Derick Beckwith  
// Program Name:      Peer-to-Peer Central Server  
// Creation Date:      March 13, 2005  
// Last Modified Date: April 12, 2005  
// COINS Course:       COINS 332 Applied Networking  
// Grade Received:     A  
// Purpose:            Set up a Central P2P Server that will receive  
//                     a connection from a client, process requests  
//                     from the client and send responses back.  
// Filename:           P2PCentralServer.java  
// -----  
  
import java.io.*;  
import java.net.*;  
import java.util.*;  
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;  
import javax.swing.event.*;  
  
public class P2PCentralServer extends JFrame  
{  
    private JTextField enterField;  
    private JTextArea displayArea;  
    private Server server;  
    private int current_user;  
    private int counter = 1;  
  
    /** Creates a new instance of P2PCentralServer */  
    public P2PCentralServer()  
    {  
        super( "P2P Central Server" );  
        // create displayArea  
        displayArea = new JTextArea();  
        Container container = getContentPane();  
        container.add( displayArea, BorderLayout.CENTER );  
        container.add( new JScrollPane( displayArea ), BorderLayout.CENTER );  
        setSize( 300, 600 );  
        setVisible( true );  
        server = new Server(this);  
        current_user=0;  
    }  
  
    public void display( String message )  
    {  
        displayArea.append( message + "\n" );  
    }  
  
    public void execute()  
    {  
        display( "Starting server....\n" );  
        server.start();  
    }  
  
    // main method begins execution of Java application  
    public static void main( String args[] )  
    {  
        P2PCentralServer application = new P2PCentralServer();  
        application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );  
        application.execute();  
    }  
} // end class P2PCentralServer
```

```
class Server extends Thread
{
    private ServerSocket server;
    private User users[];
    private int users_count;
    P2PCentralServer control;

    // additional variables for P2P central server capabilities
    private String peerFilesTable[][] = new String[10][2]; // String array for storing
    filenames and peer IP addresses
    private int index = 0;

    public Server(P2PCentralServer c)
    {
        super( "P2P central server" );
        control = c;
        users_count = 0;
        users = new User[ 10 ];
        try
        {
            server = new ServerSocket( 5001, 10 );
        }
        catch( IOException ioException )
        {
            ioException.printStackTrace();
            System.exit( 1 );
        }
    }

    public void run()
    {
        control.display("Server Started....\n");
        while ( users_count < 10 )
        {
            try
            {
                users[ users_count ] = new User( server.accept(), users_count );
                users[ users_count ].start();
            }
            catch( IOException ioException )
            {
                ioException.printStackTrace();
                System.exit( 1 );
            }
            users_count++;
        }
    }

    // User is an inner class that is spawned as a new thread
    // for each connecting client to the Server class
    private class User extends Thread
    {
        private Socket connection;
        private ObjectInputStream input;
        private ObjectOutputStream output;
        private int userNumber;
        private String usersIP;
        private StringTokenizer tokenizedMessage;
        private String firstToken;
        private String newFileName;
        private String peerIP;
        private String searchFileName;

        public User( Socket socket, int number )
        {
            userNumber = number;
        }
    }
}
```

```

        connection = socket;
        try
        {
            getStreams();
        }
        catch( IOException ioException )
        {
            ioException.printStackTrace();
            System.exit( 1 );
        }
    } // end User

    // get streams to send and receive data
    private void getStreams() throws IOException
    {
        usersIP = connection.getInetAddress().toString(); // retrieve IP address of
client
        usersIP = usersIP.substring( 1, usersIP.length() ); // trim off the "/" slash

        displayMessage( "Client " + userNumber + " connected, connection received
from: " + usersIP );
        // set up output stream for objects
        output = new ObjectOutputStream( connection.getOutputStream() );
        output.flush(); // flush output buffer to send header information
        // set up input stream for objects
        input = new ObjectInputStream( connection.getInputStream() );

        displayMessage( "\nGot I/O streams for client " + userNumber + "\n" );
    }

    private void displayMessage( final String messageToDisplay )
    {
        control.display( messageToDisplay );
    }

    // send message to client
    private void sendData( String message )
    {
        // send object to client
        try
        {
            //output.writeObject( "SERVER>>> " + message );
            output.writeObject( message );
            output.flush();
            displayMessage( "\nSERVER>>> " + message );
        }

        // process problems sending object
        catch ( IOException ioException )
        {
            displayMessage( "\nError writing object" );
        }
    }

    // close streams and socket
    private void closeConnection()
    {
        displayMessage( "\nTerminating connection with client " + userNumber + "\n" );

        try
        {
            output.close();
            input.close();
            connection.close();
        }
    }

```

```

        catch( IOException ioException )
        {
            ioException.printStackTrace();
        }
    }

    public void run()
    {
        do // loop and process different types of client messages
        {
            try
            {
                String message = "Connection OK";
                message = ( String ) input.readObject();
                displayMessage("CLIENT " + userNumber + " >>>" + message);
                tokenizedMessage = new StringTokenizer( message );
                firstToken = tokenizedMessage.nextToken();

                // process "SHARING" messages
                if ( firstToken.equals( "SHARING" ) )
                {
                    newFileName = tokenizedMessage.nextToken();
                    peerIP = tokenizedMessage.nextToken();

                    // place filename/IP in the String array peerFilesTable
                    peerFilesTable[ index ][ 0 ] = newFileName;
                    peerFilesTable[ index ][ 1 ] = peerIP;
                    index++;
                    message = "SHARING Message Received Ok from " + usersIP + " -> Client " +
userNumber + "\n" +
peerFilesTable[ index - 1 ][ 0 ] + " " +
peerFilesTable[ index - 1 ][ 1 ];
                    sendData( message );
                }
                // process "FIND" messages
                else if ( firstToken.equals( "FIND" ) )
                {
                    searchFileName = tokenizedMessage.nextToken();
                    for ( int i = 0; i < index; i++ )
                    {
                        if ( peerFilesTable[ i ][ 0 ].equals(searchFileName) )
                        {
                            peerIP = peerFilesTable[ i ][ 1 ];
                            break;
                        }
                    }
                    // check to see if table contains an IP entry for the filename
                    if ( peerIP.length() > 0 ) // found a match
                    {
                        message = "FIND_MATCH " + searchFileName + " " + peerIP;
                    }
                    else
                    {
                        message = "FIND_FAIL " + searchFileName + ": File Not Found";
                    }

                    sendData( message );
                    searchFileName = "";
                    peerIP = "";
                } // end if/else
            } // end try
            // catch problems reading from client
            catch ( ClassNotFoundException classNotFoundException )
            {
                displayMessage( "\nUnknown object type received" );
            }
        }
    }

```

```
        catch( IOException ioException )
        {
            ioException.printStackTrace();
            System.exit( 1 );
        }

    } while( !firstToken.equals("BYE") );

    displayMessage("Client " + userNumber + " disconnected");
    sendData( "BYE" );
    closeConnection();
    users_count--;          // after a user quits its connection, decrease the number
of users

    } // end run method

} // end class User

} // end class Server
```

Program 4: Project 2 - Perl problem #3

Class: COINS 333 – Applied Systems

Grade Received: A

Programming Language: Perl

Operating Systems: Windows

Compiler: ActivePerl 5.8

## Program 4 screenshot:



A screenshot of a Windows command prompt window. The title bar at the top reads "C:\WINNT\system32\cmd.exe". The window contains the following text:

```
Charleston: 7 people in 2 households  
WarnerRobins: 2 people in 1 household  
NorthCharleston: 11 people in 2 households  
C:\>
```

The text is displayed in a monospaced font on a black background. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.



### problem3

```
# -----
#
# Student Name:      Derick Beckwith
# Program Name:      Project 2, Problem #3
# Creation Date:      December 4, 2006
# Last Modified Date: December 5, 2006
# COINS Course:       COINS 333
# Grade Received:     A
# Purpose Description:

    Open and read a file named "num3.txt".

    Each line is of the form:      name city count

    which represents the household (family) name, their city of origin,
    and the number of people in that household.

    For example:

    Underwood Charleston 5
    Underwood WarnerRobins 2
    Smith NorthCharleston 3
    Anderson Charleston 2
    Jones NorthCharleston 8

    Names and cities will be made ONLY of alphabetic characters.
    Charleston and charleston should be considered different cities.

    Your code should print out the total number of people in each city
    listed in the file and the total household per city.
    You do NOT know the cities in advance.
    Example output for the above would look like:

    Charleston: 7 people in 2 households
    WarnerRobins: 2 people in 1 household
    NorthCharleston: 11 people in 2 households

    Notice household is plural only when needed. For simplicity,
    read the entire file into an array and then iterate through the
    array exactly ONE time.
    Bonus if you line up columns appropriately.

# Filename:          problem3.pl
#
# -----

$filename = "num3.txt";
%peopleInCities = (); # hash for storing the total people in each city
%familiesInCities = (); # hash for storing the total families (households) in a city

# retrieve the data from a text file.
open (FILE, $filename) or die "can't open $filename$!\n";

@arr = <FILE>; # load the whole file into an array

close FILE;

# iterate through the array a single time, populating both hashes as we go.
foreach $element (@arr)
{
    # each record should match the regex
    ($name, $city, $number) = $element =~ m#([A-Z]+) ([A-Z]+) (\d+)#;
```

```

                                problem3
# check to see if we already added this city.
# only need to check one hash as they both should have the same keys.
if ( exists $peopleInCities{ $city } )
{
    # add the number of people in the current household.
    $peopleInCities{ $city } += $number;
    # record another occurrence of the same city.
    $familiesInCities{ $city } ++;
}
else
{
    # add the new city and set it to the number of people in the current
household.
    $peopleInCities{ $city } = $number;
    # record the first occurrence of this city.
    $familiesInCities{ $city } = 1;
}
}

# print the summary
foreach $key (keys %peopleInCities)
{
    print "$key: $peopleInCities{$key}";

    # check for singularity of people in a city
    if ($peopleInCities{$key} == 1)
    {
        print " person";
    }
    else
    {
        print " people";
    }

    print " in $familiesInCities{$key} household";

    # check for singularity of families in a city
    if ($familiesInCities{$key} != 1)
    {
        print "s";
    }

    print "\n";
}

```

## Ethics Paper 1: Violence in Video Games

Class: COINS 431 – Operating Systems

Grade Received: B

## Violence in Video Games

Derick Beckwith

Violence in video games is a topic that is receiving increased media coverage and attention from politicians. With more violent game titles available to children than at any other time in history, the debate continues over who should have the authority to regulate children's access to such violent content. Both state and federal legislators have introduced bills that have attempted to enforce penalties including fines and imprisonment to violators who sell or rent violent video game titles to minors.

The current rating system used for video games in the U.S. is administered by the Entertainment Software Ratings Board known as the ESRB. The ESRB was formed in 1994 after controversy stemming from violence and gore present in games such as *Mortal Kombat* and similar fighting games prompted federal legislators led by Senator Joe Lieberman to pressure the game industry into a self-regulatory ratings model that produced the ESRB rating system (Paprocki). This system is still in use today. Although the ESRB sets standards as to how games are rated, the enforcement of these standards to restrict access to violent games is left largely to the voluntary enforcement policies of retailers who sell or rent video games (Stevens).

Recent legislative action aimed at implementing state level or national enforcement of the ESRB ratings has focused on preventing games rated with an M rating (for Mature) or an AO rating (for Adults-Only) from being sold at retail stores to children under the age of 17 or 18 respectively (Stevens). Some of the more notable bills have been passed into state law only to be later struck down by the federal district courts as unconstitutional in their restriction on children to exercise their First Amendment rights related to freedom of the press (Wikipedia). One of the most recent court rulings was made just last month in the decision by federal district court judge Ronald Whyte to declare unconstitutional a law passed by the California state assembly in October 2005 that would have made it a felony to sell or rent video games that carried an M or AO rating to a minor (Tanner).

One of the most commonly cited reasons the recent laws have been overturned is the inability of the proponents of these laws to establish an indisputable link between the acts of virtual violence portrayed in video games to acts of real physical violence performed by those who play video games. To the contrary, studies have consistently shown that people who commit acts of violence in the real world have no more attraction to violent video games than the average law abiding individual (Wikipedia). I feel that it is this fact that mostly renders laws

banning the sale of certain video game titles on the basis of their violent content as unnecessary at best and invasive at worst.

The fundamental question raised over these concerns is who should have the authority to regulate what video game content children are exposed to. At issue is the well-being of children and the desire to protect them from imagery and depictions of violent behavior that can distort their sense of reality and moral standards. The one aspect of this debate the law cannot effectively address is the role parents have in this authority structure.

The simple fact is that God placed parents in the position of authority over their children because this is how God designed the human family unit to operate. The best way to regulate children's access to violent content in video games is to leave this decision up to the responsibility of parents. Anything more restrictive is a direct assault on the God-ordained authority given in scripture to parents. To imply that a system of laws to protect children from violence in video games is more effective than an involved parent is to imply that the law presents a better way than the plan God has already put in place. Even if this were not the case, all the laws in the world could not protect a child from being exposed to inappropriate content if that child's parents are not actively engaged in the child's moral, mental and spiritual development. So to enact further legislation or rating systems would be ineffective without the enforcement of these systems by parents who already are trying their best to protect their children from harmful content. It is this apparent disconnect between the intentions of laws and their effective consequences that discourages me from lending any support for their enactment. They simply will not work.

Furthermore, I do not believe that politicians have anything remotely close to a genuine interest in the safety and well-being of my children, whether it comes to playing a certain video game or using a car seat or any other decision that has a direct impact on the child's safety. What politicians are concerned with is how they can reach into my family's life to harness a measure of control over us. It is not the case that politicians are evil people who set out on a course of world domination. Rather, it is the seduction that power over the lives of others brings that gives way to good-willed intentions which only lead to more restrictions on personal freedoms and an overall decreased quality of life. What begins as a good intentioned measure almost always turns into an overreaching premise that the state, and not I, knows how best to conduct my personal business. These actions are conducted in small incremental steps, so as not to alarm the individuals involved, but always lead in the direction of fewer freedoms and less authority over one's own family and more authority and freedom for the state to enact further restrictions.

Works Cited

Paprocki, Matt. "Video Game Violence: A History." www.googobits.com. (August 01, 2005). September 4, 2007.

<http://www.googobits.com/articles/p0-1854-video-game-violence-a-history.html>.

Stevens, Tim. "A Recent History of Video Game Legislation." Family Media Guide. (January 11, 2006). September 4, 2007.

[http://www.familymediaguide.com/features/articles/A\\_Recent\\_History\\_of\\_Video\\_Game\\_Legislation\\_Part\\_1-230.html](http://www.familymediaguide.com/features/articles/A_Recent_History_of_Video_Game_Legislation_Part_1-230.html).

Tanner, Adam. "Judge Blocks California Video Game Law." PC World. (August 07, 2007). September 4, 2007.

<http://www.pcworld.com/article/id,135603-page,1/article.html>.

Wikipedia. "Computer Game Violence." Wikipedia. (2007). September 4, 2007.

[http://en.wikipedia.org/wiki/Video\\_game\\_controversy](http://en.wikipedia.org/wiki/Video_game_controversy).

Ethics Paper 2: *Extreme Makeover: Home Edition*

Class: COINS 330 – Computer Architecture

Grade Received: A

This episode of *Extreme Makeover: Home Edition* follows the life experiences of the Tipton-Smith family. Their story is one of tragedy, loss and rebuilding. Faith Tipton-Smith is a single mother raising 2 girls, 16-year-old Missy and 8-year-old Emily, and a son, 16-year-old Ransom. After working for 24 years, Faith is finally able to purchase a home for her family in July 2004. Sadly, only 9 months later on February 4, 2005, their new home caught fire and burned to the ground. The Tipton-Smith family lost all their family photographs and their dog, Sugar, in the fire. Throughout this time, Ransom was a huge support, helping to keep the family going. He took on the responsibility of getting his sisters to after-school activities. He even helped to design the family's new house that would be built with the insurance money. Construction began and the house was framed. But the insurance funds were depleted before the home was finished. Only 3 months later on May 4, 2005, Ransom was killed in a tragic car accident right in front of the family's home. His sister Missy, who was a passenger in the car, was not seriously injured. The producers and cast of *Extreme Makeover: Home Edition* were deeply moved by the experiences of this remarkable family and decided Faith and her girls were to be the recipients of an extreme makeover.

When Ty Pennington and the rest of the show's cast and crew show up at the family's rented house, Faith shows him a photograph of a smiling Ransom that is prominently displayed on a living room wall. She walks up and kisses the glass on the picture frame. She explains to Ty that she has developed this ritual of kissing his picture every day when she comes home from work. It is obvious that Faith still misses Ransom very much. She cannot speak about him without being overcome with emotion, crying often and gazing at his photograph while speaking. In empathy, Ty also tears up when Faith describes how full of life her son was and how much he is still a part of her.

Faith, Missy and Emily are then sent off to a week of vacation in Orange County, CA. Unlike some other episodes of this show I have seen, not much time was allocated to showing the family on vacation. Their vacation is also not really portrayed as being very fun or exciting. Faith and the girls are seen walking down Hollywood Blvd snapping photos and getting pedicures. These two scenes looked very staged and artificial. It was almost like the family really did not leave their hotel room all that much and just stayed there waiting to come home. Or maybe they really did have a great time but the producers just wanted it to look this way, trying to create a mood of somberness over the loss of Ransom that would have been interrupted too much by a time of cheerfulness and light-hearted fun. I can only imagine that Faith and the girls may have felt a little out place so far from home and just wanted to return soon to see the progress made on their new house.

The part of the show that was most gripping was when Faith met with the 21-year-old woman who received Ransom's heart. Her name was Rachel Ball. Rachel and her parents were filled with gratitude toward Faith and Ransom. It was because of him that Rachel was alive. Ransom also donated his lungs, liver and pancreas to other people in need. It was wonderful to see how even in death Ransom touched the lives of many people by being an organ donor. Some of the crew from the show made a slideshow from photos of Ransom. This was shown set to music at a ceremony to honor his life. It was a great way to remember him and I think it brought healing to his mother and sisters.

The rest of the show is about how the family's new home is designed and constructed by some very bright and creative designers. The crew does a great job of completing the house in one week and, as usual, everything looks fabulous on camera. Missy's room is decorated with custom wallpaper, on which an article she wrote for the school newspaper is displayed. Emily's room is designed around her piano playing interest, complete with a wrap-around bed post made of big piano keys and her very own grand piano. My favorite part of the house was the customized home theatre room that came with a popcorn machine in the corner. One of the more meaningful things Ty and the cast did was to make a metal sign with the words "SLOW DOWN" engraved on it. Faith wanted the sign to be placed at the scene of Ransom's crash to hopefully prevent other traffic deaths. I could see it meant a lot to Faith to have this sign where it could warn other teenagers like Ransom to be careful. When the family returns from vacation and sees the new house, they are hysterical in their happiness and near disbelief at what they are experiencing. I am certain that much of the external excitement from the crowd and cast members is pumped up by the producers but the family's emotional reaction is very genuine and sincere.

It is at this point in the show that I am fascinated to see how people are going to react at being given such a wonderful gift. Faith is understandably very excited about having the new home and appreciative of the builders and community that pulled together to pay off her mortgage. But there is also a streak of materialism and sense of selfish entitlement that has crept into this woman's attitude. Through her tears of joy, she declares to all standing there that she has been waiting a long time for this moment and "deserves" this to happen to her and her family. While I can certainly empathize with her situation and agree this a great new start for her family, a lack of a true outward expression of gratitude to God in this time of great material blessing is a sign of her own lack of spiritual grounding. It is not so much an indictment of her as it is an example of the attitude of entitlement and materialism that is pervasive throughout our culture. I am not excluding myself from this in any way; I know I am guilty of the same attitude. But without a personal relationship with the Lord and a Biblically-based understanding that all things in life come from Him, it is only natural for a person in this culture to be have a similar reaction as Faith had.



Ethics Paper 3: Will Self-Aware Robots Ever Become a  
Reality?

Class: COINS 480 – Artificial Intelligence

Grade Received: A

## Will Self-Aware Robots Ever Become Reality?

Derick Beckwith

Can we really program a computer to have a human-like consciousness? I do not think this is possible with the knowledge we have today. I agree with Raj Reddy who said we do not even yet have the necessary “conceptual breakthroughs.” And if we ever did, there would need to be an enormous amount of research funding to bring it to reality (Reddy). There are some computers that can mimic some human qualities like Eliza and some that can solve complex problems like Deep Blue but no computer yet exists that can simulate all aspects of a person. The Turing test has yet to have a clear winner.

If there ever comes a day when artificial intelligence has reached the point of being able to fully simulate humans it will still be that, a simulation not unlike the holodeck in the television series *Star Trek – The Next Generation*. In the more distant future, say 200 years from now, there may even be machines that physically simulate human behavior like the human-looking robots from *The Terminator* movie series. I could envision this type of simulated human behavior becoming a reality but it would still not have a consciousness like real people. It would still be a collection of silicon, electricity and maybe some carbon-based bio-computers running highly sophisticated software. They may even be indistinguishable in appearance from humans.

One example of a “cognitive system” is in development now at DARPA. This project is an attempt to combine speech recognition, machine learning and other “bits and pieces” of technology to construct a sort of intelligent warfare system. This system would be able to learn and adapt to changing scenarios on a battlefield and respond in much the same way as a human soldier would (Melymuka). Although a project like this is a significant step towards an

intelligent system for military application, it is not anywhere close to becoming a self-aware computer.

I think the near future of intelligent systems will be more like *Robocop* than *The Terminator*, artificial intelligence combined with some parts of a living human into a sort of human cyborg that is both human and machine (Rubin). The need for this will likely evolve from current advances in the use of robotic prosthetics for wounded and dismembered combat veterans. Participation would be voluntary but I can see a day when hardware and software devices are used to augment the human body in this way. This would combine the best of both worlds, human reasoning coupled with the speed and accuracy of a computer, a helper system like the one being developed at DARPA and perhaps a robotic body capable of much more speed and agility than a “normal” human. And considering that human longevity is largely limited by the lifespan of the heart, lungs and other major organs, what promises could hold for an artificial replacement for these that would nourish and be controlled by the original brain and nervous system. In this protected “life support”, people may live for who knows how long!

The other part of the question deals with *should* we ever program a computer to have consciousness. Although this question raises a hypothetical situation it is an interesting one nonetheless. Way back in Genesis 1:26 and 1:28, God decided to let man “rule over . . . every living thing that moves on the earth.” In Genesis 2:7 it also says that God breathed into Adam the breath of life and he became a “living being”. So we have mankind being put in place as the ruler of all the living things on earth and this act of God giving life to mankind. If it was ever possible to program consciousness into a computer, it would still not be life in the same way that God gave life to Adam or how He gives life at conception. Thought of another way from the entropy point of view, it is against the notion that the universe moves from a state of order to

disorder if a human could ever design, create and build something that is both living and also more advanced in *every* way. To do so would also violate the order God established with mankind being over all the other “living” things on earth. With this premise in place, the answer to the question of should we program a computer to have consciousness is yes, because although it cannot be done, the process will undoubtedly reap many benefits for all of us “real” people. Some may say it is dangerous to use absolutes like “it can never happen” in arguments like these but although I agree that “not everything that is imaginable is possible”, everything that *is* possible has yet to be imagined (Rubin).

## Works Cited

Melymuka, Kathleen. "Good Morning, Dave. . . ." ComputerWorld. (November 11, 2002).

February 7, 2008.

<http://www.computerworld.com/softwaretopics/software/appdev/story/0,10801,75728,00.html>

Reddy, Raj. "Infinite Memory and Bandwidth: Implications for Artificial Intelligence."

KurzweilAI.net. (February 22, 2001). February 7, 2008.

<http://www.kurzweilai.net/meme/frame.html?main=/articles/art0066.html%20>

Rubin, Charles. "Robots 'R' us?" *The Pittsburgh Post-Gazette*. May 14, 2006.

<http://www.post-gazette.com/pg/06134/689745-109.stm>

The Holy Bible, Genesis Chapters 1 and 2, New American Standard Version.

<http://www.blueletterbible.org/cgi-bin/tools/printer-friendly.pl?book=Gen&chapter=001&version=nasb>

## Presentation 1: Linux Process Scheduler

Class: COINS 431 – Operating Systems

Grade Received: A

# Linux Process Scheduler

Derick Beckwith



# Overview

- Linux scheduling basics.
- No SMP considerations.
- Introduce data structures and algorithms used by the various scheduler implementations.
- Kernel versions 2.4, 2.6.8.1 and 2.6.23 use radically different schedulers.



# Linux Scheduling Basics cont.

- Linux makes no distinction among processes and threads, these are all just “tasks” as far as the kernel and scheduler are concerned.
- A process is preempted when the execution of the process is interrupted in order to run a different, higher priority process
- Scheduler only preempts processes running in user space, kernel space processes are not preempted (this avoids the associated synchronization problems)

# Linux Scheduling Basics

- Real Time - Extremely high scheduling requirements, usually the highest priority process in a system.
- Batch – lower priority background tasks.
- Interactive – higher priority tasks, perceive response to user an important consideration.



# Linux Scheduling Basics cont.

- CPU time is divided into units called *epochs*.
- Each process is assigned a time quantum at the start of the epoch.
- A process can be scheduled several times in the same epoch until its time quantum is up.
- An epoch ends when all processes have exhausted their assigned time quanta.
- The `nice()` system call manipulates the value of the given process's time quantum.

# Scheduling Priority

- Static – Real time processes, value from 1 to 99 set by user.
- Dynamic – conventional processes, value assigned by the scheduler.
- Scheduler will only start running conventional processes when there are no real time processes to run.



# When does scheduling occur?

- The currently running process's time slice has expired.
- A higher priority process is ready to run.
- A kernel process yields the CPU explicitly.
- An interrupt handler finishes executing its code.
- A system call returns to user space.

## 2.4 Overview

- 1400 Lines of code
- Relatively simple implementation.
- Does not scale well to large numbers of processes.
- Running time of  $O(n)$  for  $n$  runnable processes being scheduled.
- Iterates over each process to set priority level, pick next process to run, calculate new time quantum.



## 2.4 Data Structures

- There is one run queue, and it's a linked list of process descriptors.
- A process descriptor contains fields related to that process:
  - `Need_resched` – flag to invoke `schedule()`
  - `Policy` – scheduling class (RR, FIFO, etc)
  - `Counter` – remaining CPU ticks left in the current time quantum

# How `do_fork()` is bounded

- When `do_fork()` is called, the counter field of the parent and child process is set so as to not allow a process to hog all the CPU cycles by forking more processes.
- This is implemented by splitting the parent's time slice in two and assigning each half to the parent and child.

```
current->counter >>= 1;  
p->counter = current->counter;
```



## The 2.4 `schedu1e()` function

- `Schedu1e()` is the function in the Linux kernel that does the actual scheduling.
- Runs when a new process needs to be selected for scheduling.
- Is called when the currently running process is blocked, waiting for a resource.

# Picking the best candidate to run

```
list_for_each(tmp, &runqueue_head)
```

```
{
```

```
    p = list_entry(
```

```
    tmp, struct task_struct, run_list);
```

```
    if (can_schedule(p))
```

```
    {
```

```
        int weight =
```

```
        goodness(p, this_cpu,
```

```
        prev->active_mm);
```

```
        if (weight > c)
```

```
            c = weight, next = p;
```

```
}
```



# Assigning new time quantum

```
if (!c)
{
    for_each_task(p)
    p->counter =
        (p->counter >> 1) + p->priority;
}
```

# Drawbacks of 2.4 improved in 2.6

- Scalability
- Large average time slices
- I/O bound task priority boosting
- Real time application support



## 2.6 Overview

- 5700 Lines of code
- Run queue and priority arrays are the basic data structures
- One run queue per processor
- Two priority arrays per run queue
- All algorithms in the scheduler perform in  $O(1)$ , or constant, running time.

# Run Queues

- Foundational data structure upon which the scheduler is built.
- Keeps track of all runnable tasks on a CPU.
- Maintains the CPU's special thread state information.
- Manages the two priority arrays.



# Run Queue Data Members

```
spinlock_t lock;  
unsigned long nr_running;  
unsigned long long nr_switches;  
unsigned long nr_uninterruptible;  
unsigned long expired_timestamp;  
unsigned long long timestamp_last_tick;  
task_t *curr, *idle;  
struct mm_struct *prev_mm;  
prio_array_t *active, *expired, arrays[2];  
int best_expired_prio;  
atomic_t nr_iowait;
```

# Why use Priority Arrays?

- Allows the scheduler's algorithm to find the task with the highest priority in constant time.
- Makes RR scheduling in same priority level a constant time problem also (circular queue).
- Using two priority arrays in unison to hold active and expired processes allows for initializing a new epoch in constant time (update 2 pointers each epoch).
- Alleviates the need for iterative constructs.



# Priority Array

```
struct prio_array {  
    unsigned int nr_active;  
    unsigned long bitmap[BITMAP_SIZE];  
    struct list_head queue[MAX_Prio];  
};
```

# How Priority Arrays Work

- Implemented as an array of linked lists of runnable processes at each priority level.
- Uses a bitmap to efficiently discover the highest priority process.
- When a task with priority 10 becomes runnable bit 10 in the bitmap gets set to 1



# How Priority Arrays Work cont.

- There is one list in the array for each priority level.
- When a task is added to a priority array, it is added to the list within the array for its priority level.
- The highest priority task in a priority array is always scheduled first.
- Tasks within a certain priority level are scheduled round-robin.



## The 2.6 `schedu1e()` function

- Checks the run queue for runnable task.
- If there are runnable tasks in the runqueue but not in the active priority array, then the active and retired priority arrays are swapped. (begin new epoch)
- Next, the active priority array's bitmap is checked to find the highest priority level with a runnable task.

# Real time differences

- 2.6 provides soft real time support.
- Real time processes will preempt regular processes.
- Real time priorities are set statically.
- Not all versions of the 2.4 kernel can offer any real time guarantees.
- Not all versions of the 2.4 kernel offer preemption.
- Priority inversion occurs frequently in the 2.4 kernel.



# Completely Fair Scheduler

- Merged into the 2.6.23 kernel release.
- Complete rewrite of the 2.6.8.1 version.
- Does not use run queues, time slices or priority arrays.
- Uses red-black trees.
- Runs in  $O(\log n)$ .

# CFS goals

- Simplify the algorithm used in 2.6.8.1.
- Replace time slices with accounting at the nanosecond granularity.
- Utilize concept of “sleepers fairness”.
- Trend toward using modular schedulers.



# References

- Aas, Josh. "Understanding the Linux 2.6.8.1 CPU Scheduler." Silicon Graphics, Inc. February 17, 2005. 26p.  
<http://whitepapers.techrepublic.com.com/whitepaper.aspx?docid=289299>
- Aivazian, Tigran. *Linux Kernel 2.4 Internals*. Veritas Software, Ltd. August 7, 2002. 82p.
- Bovet, Daniel and Cesati, Marco. *Understanding the Linux Kernel*. 1<sup>st</sup> Ed. O'Reilly Publishing, 2000. 704p.
- Corbet, Jonathan. "Kernel space: Fair user scheduling for Linux." LinuxWorld.com. October 23, 2007.  
<http://www.linuxworld.com/news/2007/101207-kernel.html?fsrc=rss-linux-news>
- Love, Robert. *Linux Kernel Development*. Sams Publishing, 2003.  
<http://www.informit.com/articles/article.aspx?p=101760&seqNum=2&rl=1>
- McCabe, Michael. "The Linux Scheduler 2.4 vs 2.6." [mccabemt@clarkson.edu](mailto:mccabemt@clarkson.edu).
- Wikipedia, the free encyclopedia. "Completely Fair Scheduler." October 28, 2007. [http://en.wikipedia.org/wiki/Completely\\_Fair\\_Scheduler](http://en.wikipedia.org/wiki/Completely_Fair_Scheduler)



## Presentation 2: rtNEAT and NERO

Class: COINS 480 – Artificial Intelligence

Grade Received: A

Place photo here

# rtNEAT and NERO

## An application of neural networks in video games.

# The need for machine learning in video games

- “Dumb” NPCs make for predictable and boring gameplay.
- Agent training is often done offline before a game is released.
- Real time machine learning opens up new possibilities in playing against “smart” agents.
- Reinforcement Learning (RL) techniques can be applied to games.

# Games present challenges for traditional RL techniques

- Large state/action space - many interactions going on that have to be evaluated in the game loop.
- Diverse behaviors - RL could make for boring agent behavior.
- Fast adaptation - games are real time, RL cannot take too long.
- Memory of past states - makes the game more interesting.



# NeuroEvolution of Augmenting Topologies

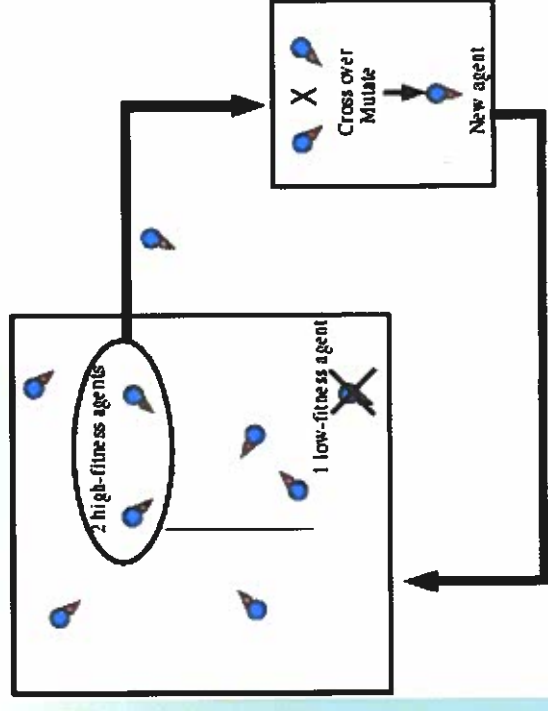
- NEAT is based on genetic algorithms that evolve increasingly complex neural networks.
- The goal is to have a population become increasingly sophisticated over generations.
- The population's behavior is evolved using reinforcement learning tasks.

## NEAT is based on 3 key ideas

- Flexible genetic coding - mutations help improve the GA results.
- Population speciation - individuals compete in their own species, not the whole population.
- Minimal starting structure - complexity is incremental and appropriate to the given problem space.

# rtNEAT runs a real time replacement cycle

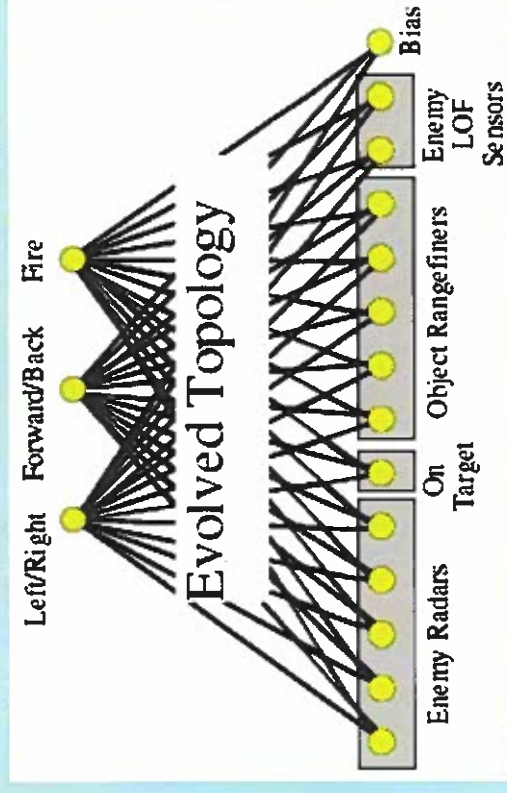
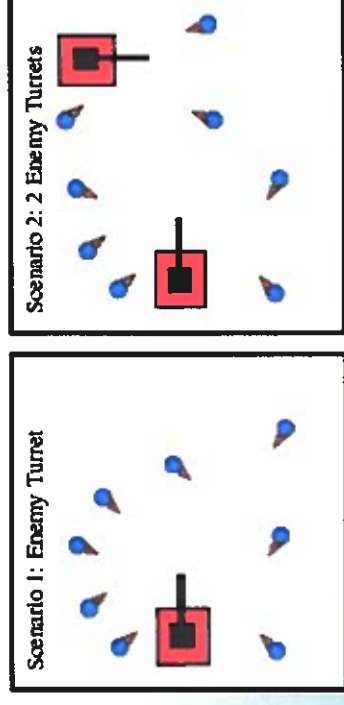
1. Remove agent with worst fitness that has lived the minimum time allowed.
2. Re-estimate fitness for all species.
3. Choose parent species for new offspring.
4. Adjust compatibility threshold and reassign all agents to species.
5. Place the new agent in the world.





# NeuroEvolving Robotic Operatives (NERO)

- Player trains a team of robots for battle.
- Robot behavior evolves in real time, allowing player to interact (teach).
- Player trains the robots using learning tasks that increase in difficulty.
- Aggressive and avoidant behavior evenly matched in battle mode.
- Examples of evolved behaviors.

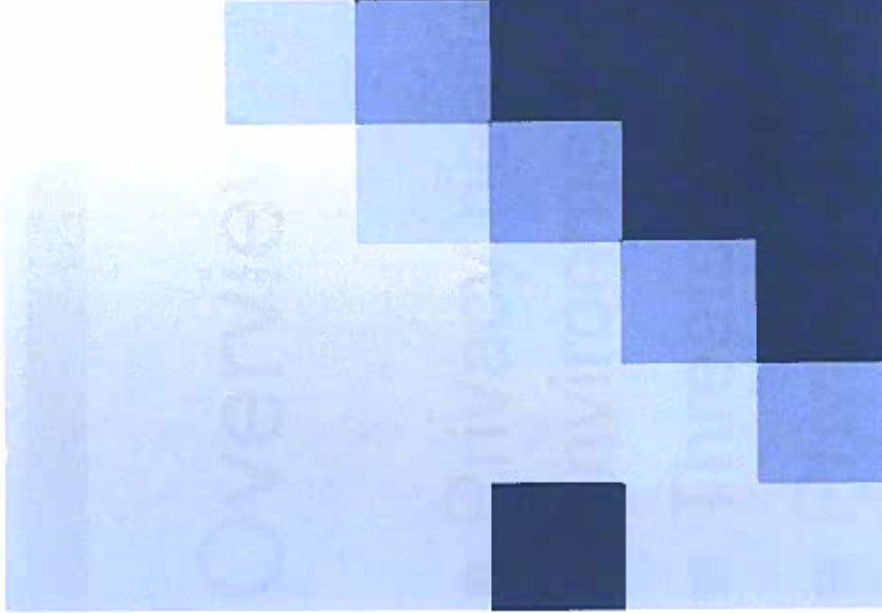




## Presentation 3: Privacy in Modern Computer Systems

Class: COINS 433 – Network Security

Grade Received: A



# Privacy in Modern Computer Systems

Derick Beckwith

April 2006



# Threats to Privacy

- Bribes
  - Public Records
  - Aggregation and Data Mining
    - Credit/Debit card transactions put you at a location at a specified time
    - Cell phone records place you at a general location, GPS even more exact
    - Electrical usage patterns indicate when you are not home
  - Credit Card fraud
-



# Threats to Privacy

- Government Threats
    - Taxation
    - Social Security
    - Infectious Diseases
    - Law Enforcement
    - Electronic Surveillance
-





# Threats to Privacy

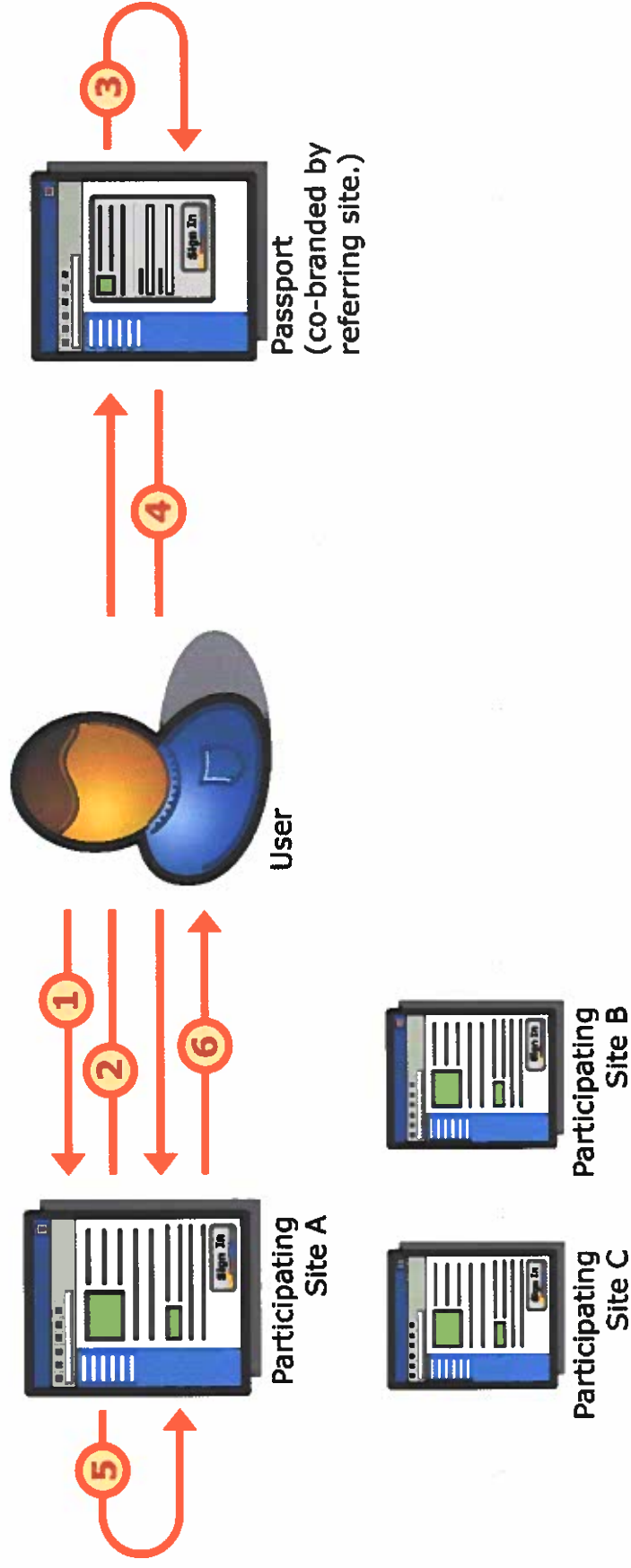
- Corporate Rights and Private Business
    - Companies can collect data a government cannot
  - Privacy for Sale?
    - Frequent-buyer cards from grocery stores, electronic retail outlets, etc offer powerful marketing tool
    - Online surveys, “free” offers, drawings
-



# Privacy Protection Controls

- Two observations:
    - Volume of collected data can be enormous
    - Potential to collect, sort and mine this data is also enormous
  - Authentication
    - Microsoft's .NET Passport
    - Common authenticators are readily accessible
-

# Microsoft's .NET Passport





# **Microsoft Passport Network Privacy Statement**

- **“We do not control or monitor the privacy practices of sites on the Passport Network, and the privacy practices of those sites will vary. However, all sites or services on the Passport Network must contractually agree to have a posted privacy statement.”**
-



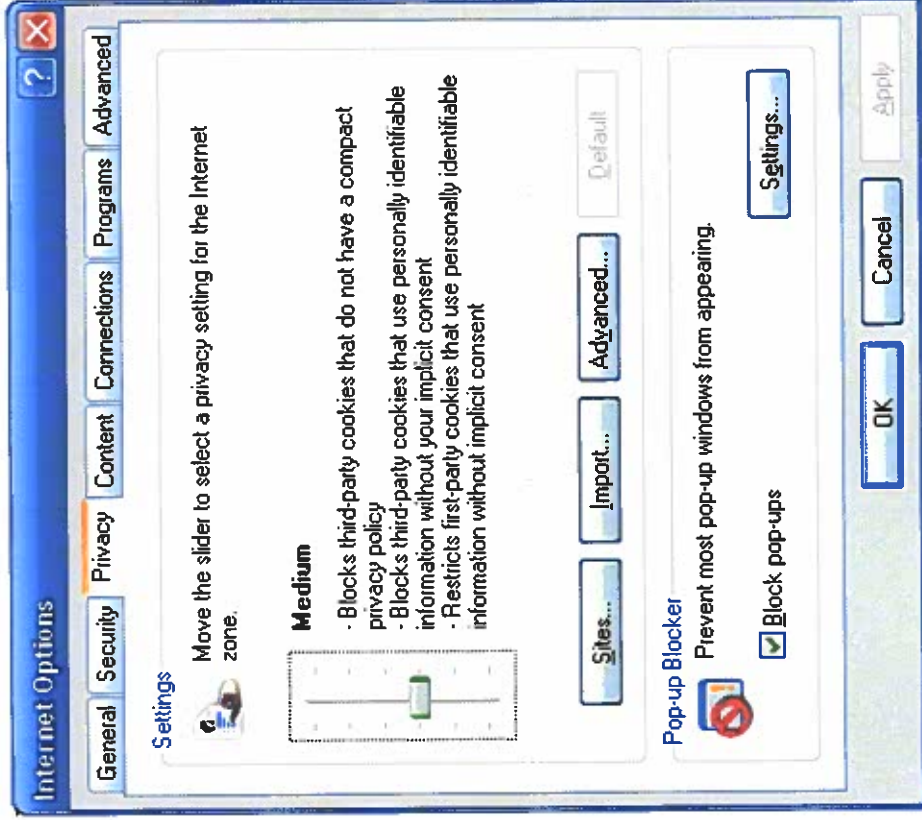
# A word on cookies

- Cookies are pieces of information generated by a Web server and stored in the user's computer, ready for future access.
  - Cookies are embedded in the HTML flowing back and forth between the user's computer and the servers.
  - Cookies were implemented to allow user-side customization of Web information.
-

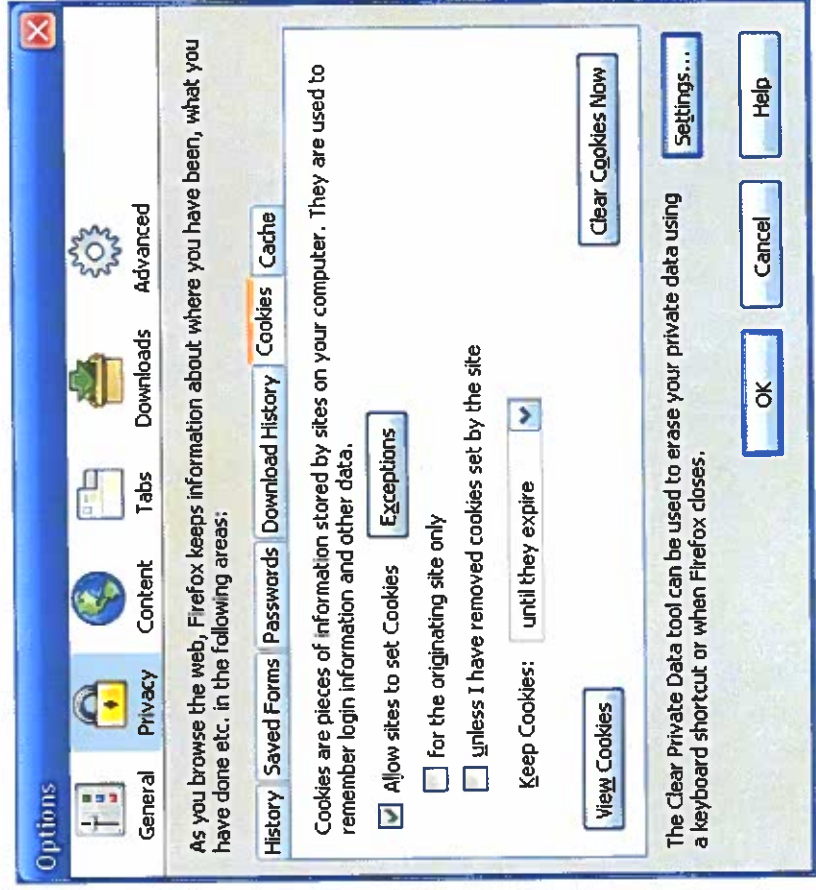


# Browser Cookies Settings

## Microsoft Internet Explorer



## Mozilla Firefox





# Privacy Protection Controls

- Anonymity
    - E-mail Anonymizers
    - Onion Routing
    - Anonymous web browsing
  - Pseudonymity
    - Swiss bank accounts
    - Web based e-mail accounts
-



# Privacy Protection Controls

- Legal Controls
  - EU Data Protection Act – 1998
  - Gramm-Leach-Bliley Act – 1999
  - HIPAA - 1996